pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

One Neuron Is a Logistic Regression

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

One model, three tools

Keras

```
Dense(1, activation="sigmoid")
```

fitted by Adam

scikit-learn

```
LogisticRegression(C=np.inf)
```

no penalty

Excel

```
a column of formulas
```

fitted by Solver

From today: fit rows and stopping rows

18,000 training rows: 14,400 to fit, a stratified fifth (3,600) to stop or choose. Validation reports once, for the model chosen.

If one neuron is a logistic regression, all three must give the same weights.

How a Dense layer works

1. Weight each input

one weight per input

2. Add up

the products plus a bias

3. Sigmoid

squeezes the total into 0 to 1

4. Units

a Dense layer is a group of neurons

You control

The inputs, the units, and the activation.

What comes out

sigmoid(bias + x @ kernel): day 1's formula, so one unit is a logistic regression.

A Dense layer of one unit is day 1's formula

```
keras.layers.Dense(1, activation="sigmoid", name="neuron")
output = sigmoid(bias + x @ kernel)
```

(2, 1)

kernel: inputs by units

(1,)

bias: the intercept

3

parameters, two inputs

Weights set by hand: predict equals the formula worked in numpy, within 1e-6.

The kernel holds the coefficients, the bias the intercept: a logistic regression.

How scaling works

1. One learning rate

for every weight

2. Different sizes

a huge step for one weight, a tiny one for another

3. Standardize

mean 0 and spread 1 for every column

4. Fit rows only

learn the mean and spread there, apply everywhere

You control

Standardizing or min-max, and which rows it learns from.

What comes out

A float32 array per set of rows, in units of standard deviations.

Scaling for a network: fitted on the fit rows only

128867.52

standard deviation of LIMIT_BAL

1.11

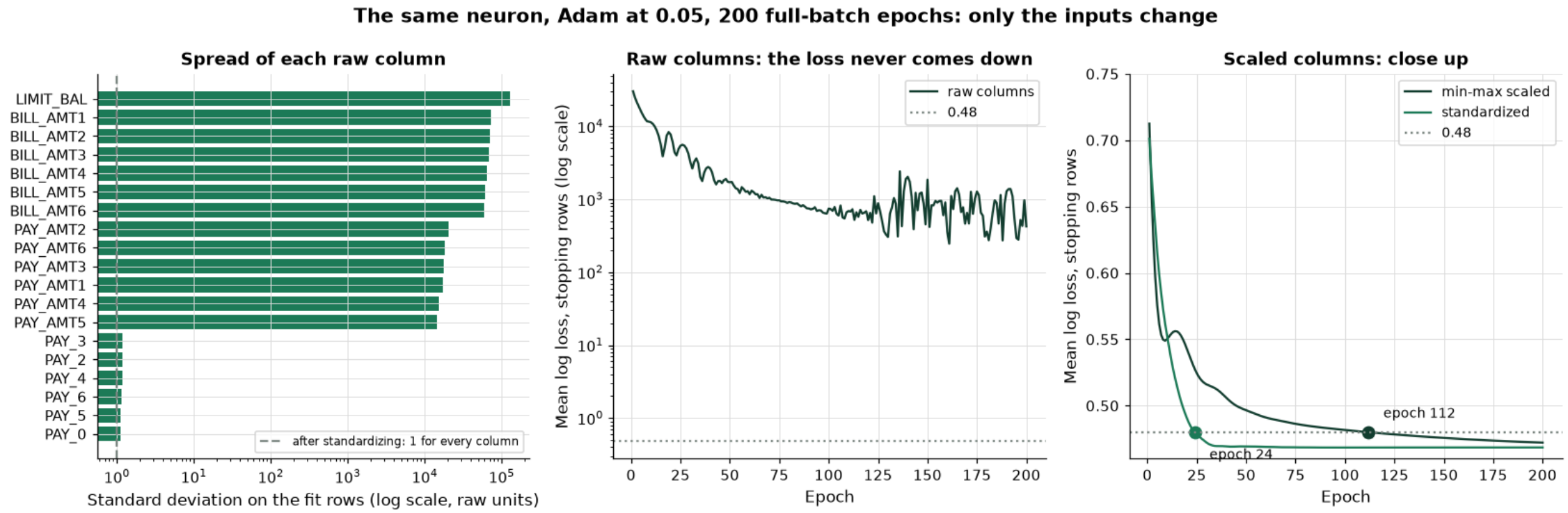
standard deviation of PAY_0

```
scaler, A_fit, A_stop, A_valid = netkit.fit_arrays(StandardScaler(), X_fit, X_stop, X_valid)
```

Scaled column	fit mean	fit sd	stopping mean	validation mean
LIMIT_BAL	0.0	1.0	0.0318	0.0061
PAY_0	0.0	1.0	0.0172	-0.0082
BILL_AMT1	0.0	1.0	0.0127	0.0292

The scaler learns from rows, so it is fitted like a model: on the fit rows, nothing else.

Scaling, side by side



Raw columns: stopping loss 30362.2422 at epoch 1, 425.9600 at epoch 200. Standardized: below 0.48 at epoch 24; min-max at 112.

How compile and fit work

1. Loss

the number to lower: binary_crossentropy

2. Optimizer

the rule that turns gradients into steps

3. Fit

batches, gradients, steps; an epoch is every row once

4. History

the loss after every epoch

You control

The optimizer and its learning rate, the epochs, the batch size.

What comes out

The fitted model, changed in place, and a History.

Compile and fit, argument by argument

```
neuron_two.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1), loss="binary_crossentropy")
neuron_two.fit(X, y, epochs=300, batch_size=len(X), shuffle=False, verbose=0)
```

Argument	What it sets
optimizer	how a step is taken: Adam, a step size per weight
loss	binary_crossentropy, day 1's mean log loss
epochs	passes over the rows
batch_size	rows per gradient; all of them is full batch
shuffle	reorder each epoch; nothing to reorder in one batch
verbose=0	no progress bar

Two columns: scikit-learn's weights

Coefficient	neuron	scikit-learn
intercept	-1.102609	-1.102607
PAY_0	0.721292	0.721293
LIMIT_BAL_10k	-0.020327	-0.020327

Adam at 0.1, full batch, 300 epochs

Mean log loss 6.2950 at epoch 1, 0.4678 at epoch 300. Equal within 1e-4.

2.0571

odds ratio, PAY_0

0.9799

odds ratio, per 10,000 of limit

Course 4 day 6's odds ratios, to 4 decimals: they describe the training rows of this file.

How Solver works

1. Target cell

the sum of the log-likelihoods

2. Changing cells

the intercept and the two weights

3. Direction

as large as possible

4. Search

recalculates the sheet as it goes

You control

The target, the changing cells, and the direction.

What comes out

The changing cells at the best point: the neuron's weights.

Solver finds the same weights

Excel side by side (tested in Excel)

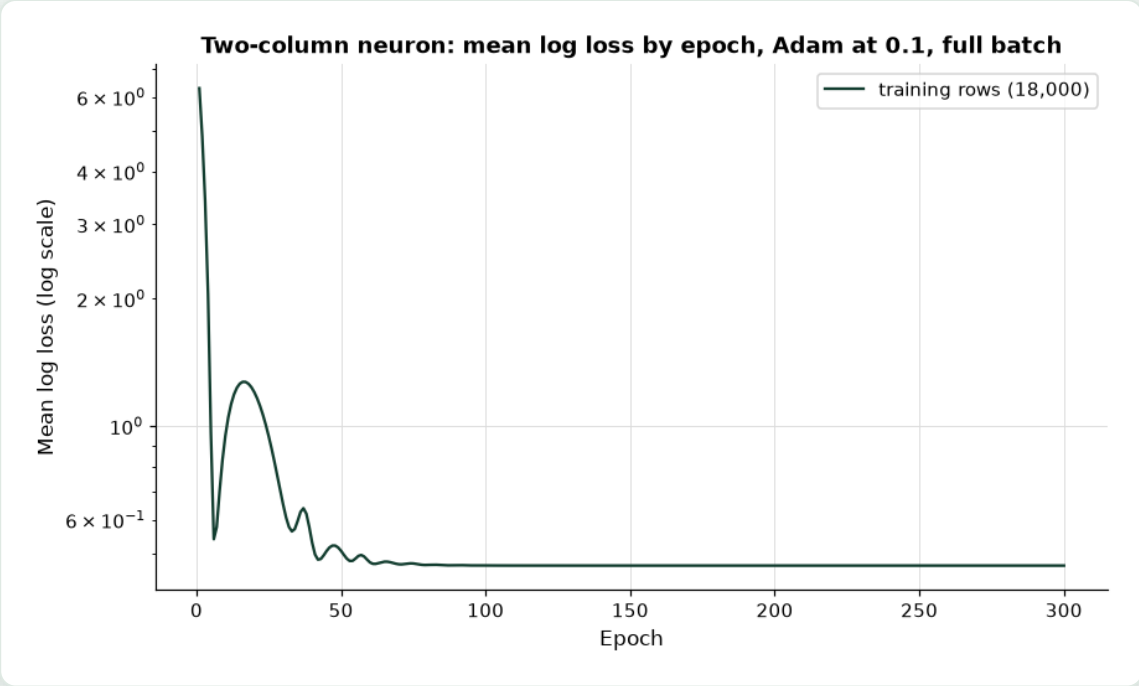
`=1/(1+EXP(-($K$1+$K$2*B2+$K$3*D2)))`	E2, filled down; K1:K3 start at 0
`=A2*LN(E2)+(1-A2)*LN(1-E2)`	F2, filled down
`=SUM(F2:F18001)`	K4, the objective
`Solver: Max K4 by changing K1:K3`	GRG Nonlinear, Course 4 day 6's options

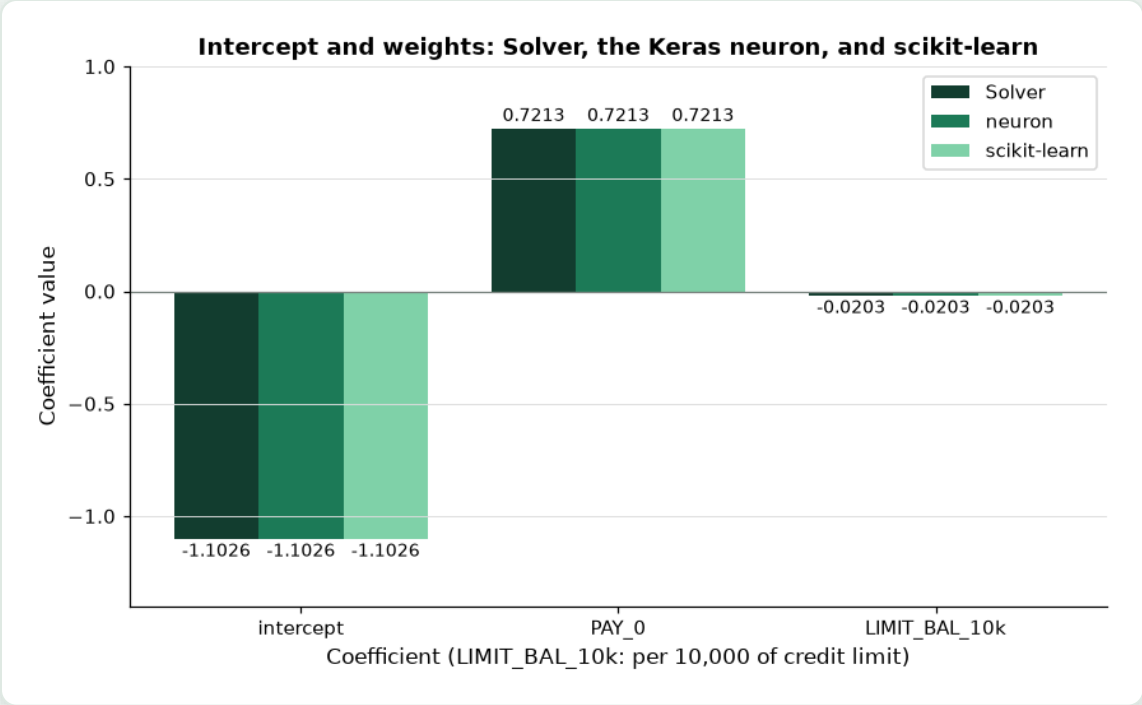
Coefficient	Solver	neuron
intercept	-1.102607	-1.102609
PAY_0	0.721293	0.721292
LIMIT_BAL_10k	-0.020327	-0.020327

Equal within 1e-4

Solver, scikit-learn, and Keras: three tools, one model.

Two columns, drawn





Loss 6.2950 at epoch 1, 0.4678 at epoch 300. Three bars per coefficient, one height: intercept -1.102609, PAY_0 0.721292.

Nineteen columns: the same minimum

0.461056

neuron: mean log loss, fit rows

0.461056

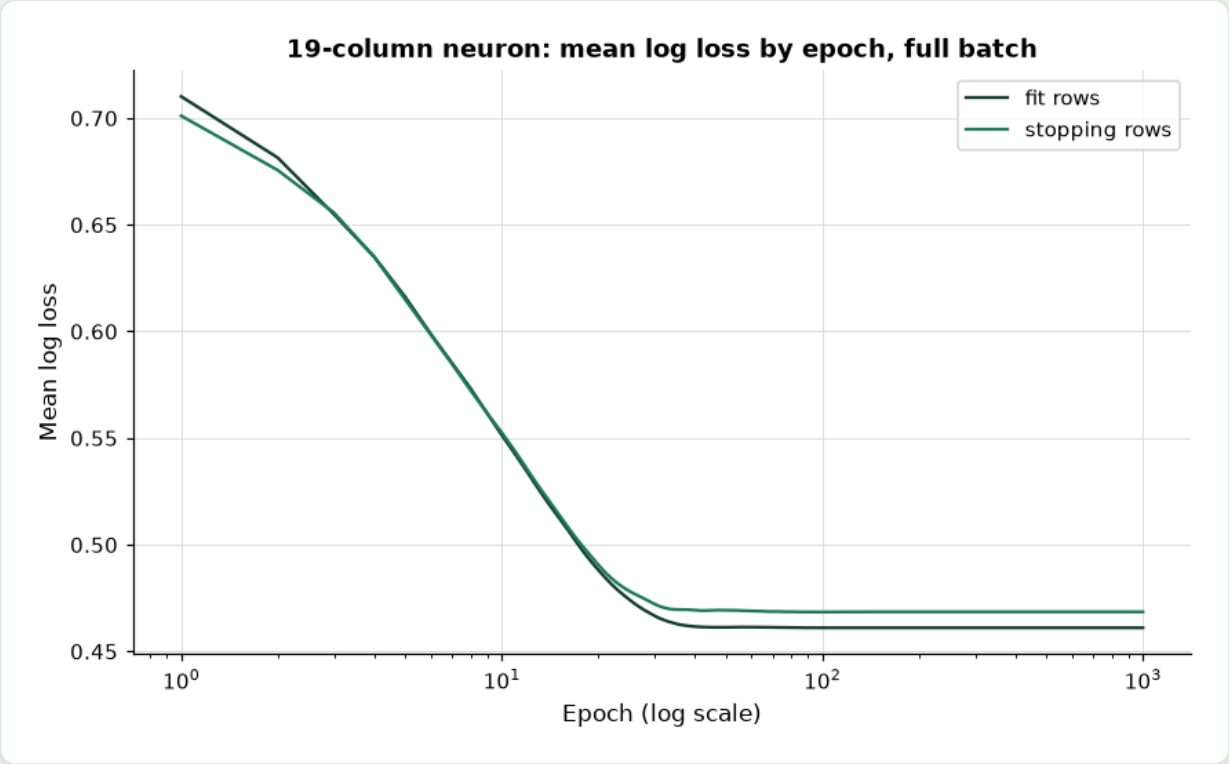
scikit-learn, same rows

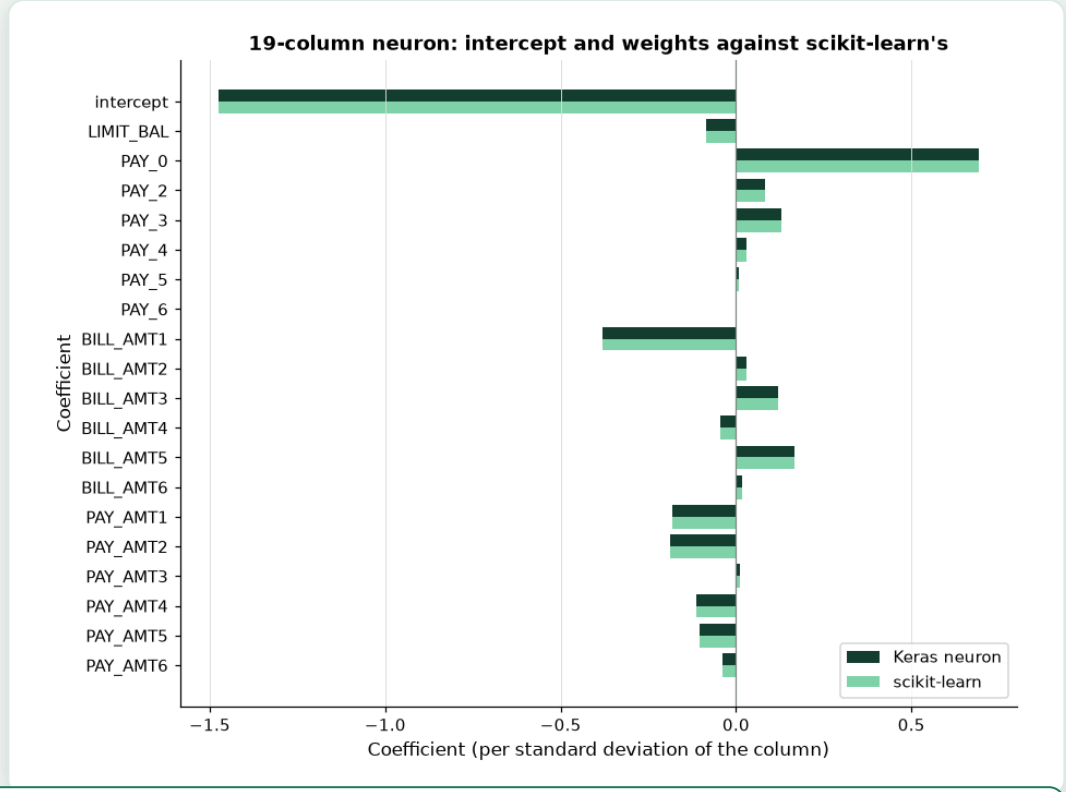
Bill weight	neuron	scikit-learn
BILL_AMT1	-0.3794	-0.3794
BILL_AMT2	0.0292	0.0292
BILL_AMT3	0.1210	0.1210
BILL_AMT4	-0.0461	-0.0461
BILL_AMT5	0.1652	0.1652
BILL_AMT6	0.0178	0.0178

All 20 coefficients equal within 1e-4

Read one at a time, the bill weights say a higher bill this month goes with a lower output. Look first at how alike the six columns are.

Nineteen columns, drawn





At epoch 1,000: fit rows 0.4611, stopping rows 0.4685. A small gap that never widens; every pair of bars the same length.

Near-copy columns: a flat loss

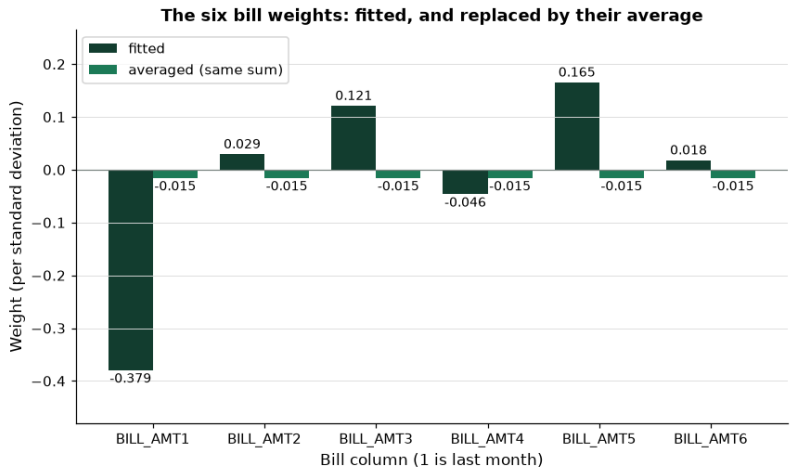
0.80 to 0.95

correlations between the six bill columns

0.3640

largest change in one weight

Mean log loss	fitted	averaged
fit rows	0.4611	0.4622
stopping rows	0.4685	0.4679



Many different weights fit almost equally well: compare networks by loss and scores, not weights.

How mini-batches work

1. Full batch

one gradient from every row: one careful step

2. Mini-batch

a gradient from a small random group, then step

3. Noisy

each estimate differs from the full gradient

4. Shuffle

new groups every epoch

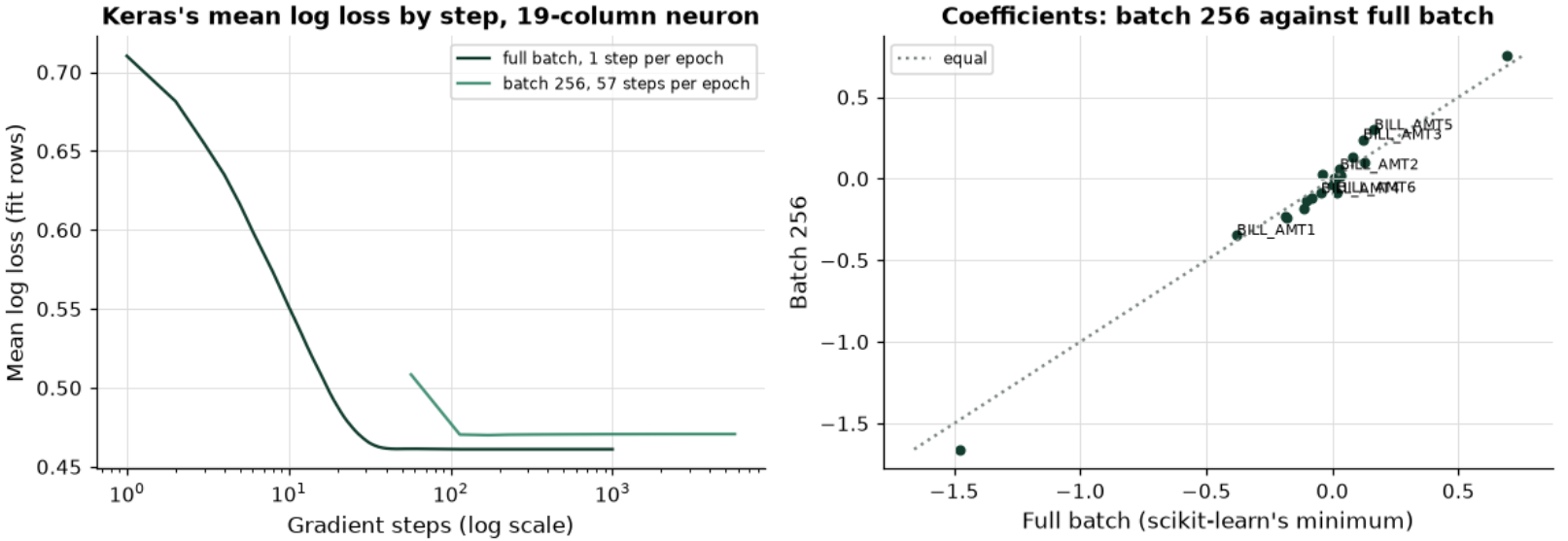
You control

The batch size and the number of epochs.

What comes out

Many more steps; near the full-batch answer, not on it.

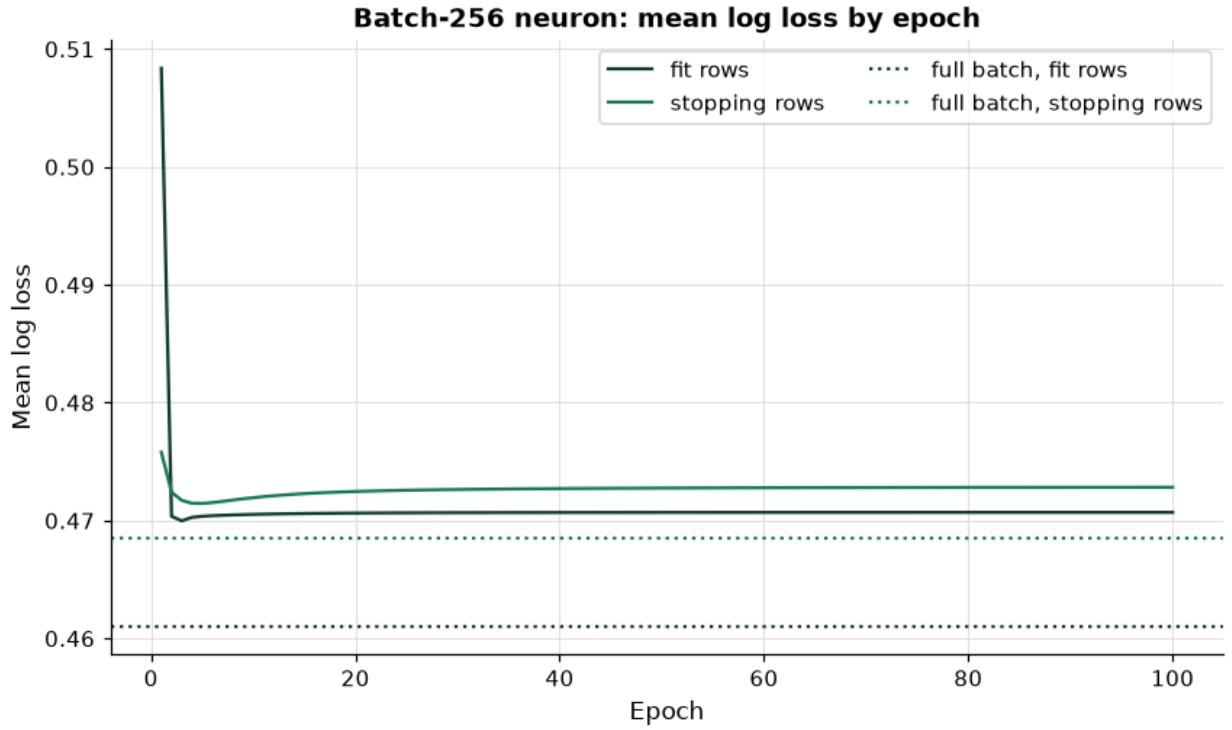
Mini-batches stop short



Batch 256: 57 steps an epoch, 5,700 in all. Coefficients up to 0.1827 off (intercept); furthest weights BILL_AMT5, BILL_AMT3, BILL_AMT6. Loss 0.4648 against 0.4611 on the fit rows, 0.4728 against 0.4685 on the stopping rows.

One setting changed: the batch size, all 14,400 rows to 256; chosen on the 3,600 stopping rows: full batch.

Mini-batch loss by epoch



Every epoch above the dotted full-batch lines: stopping rows 0.4728 at the end against 0.4685.

The chosen neuron, once on validation

Model	Rows fitted on	ROC AUC	Average precision
Neuron, 19 columns	14,400 fit rows	0.7180	0.4945
scikit-learn, unpenalised	14,400 fit rows	0.7180	0.4945
Course 4 logistic regression	18,000 training rows	0.7190	0.4946

Validation, scored once. These are the bar for day 3's hidden layers.

Allowed names

model probability of default in this file

flagged at threshold t

the model's score (class-weighted)

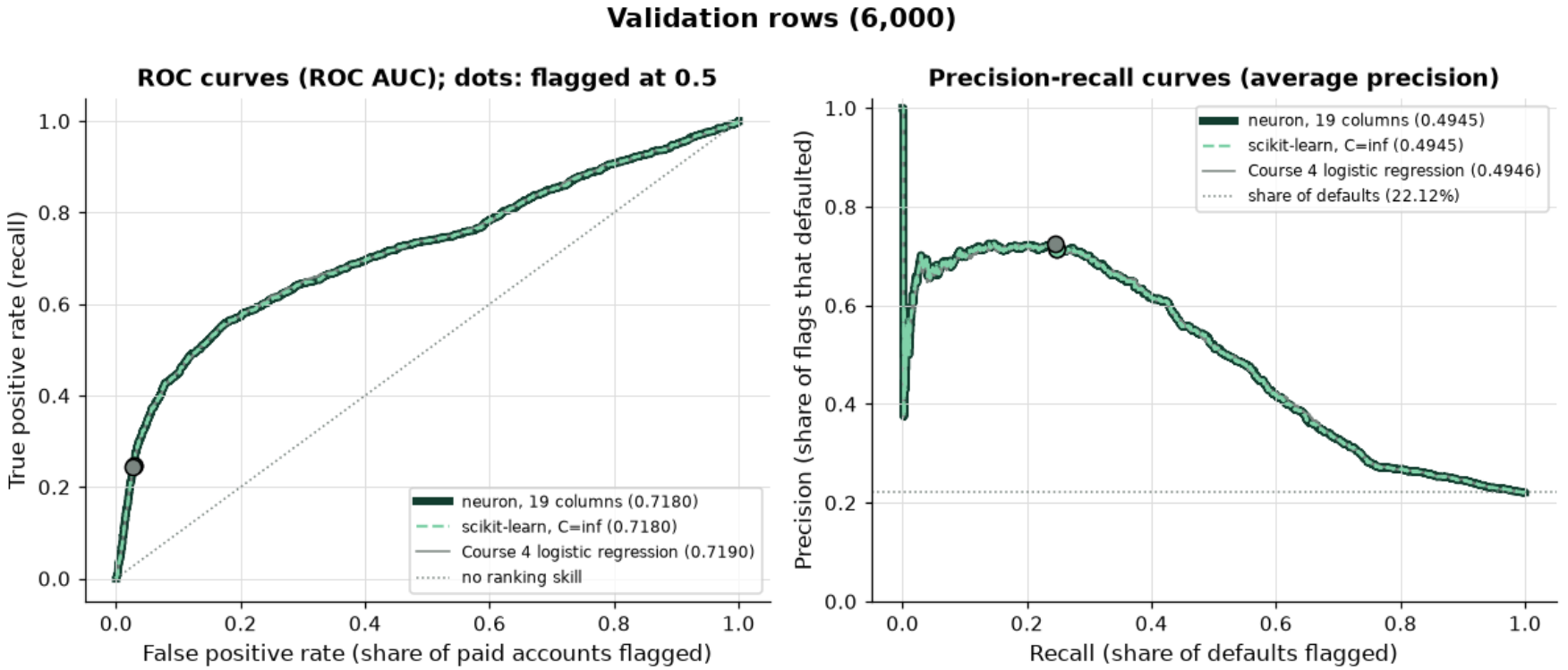
Never called

a credit decision

an approval or a decline

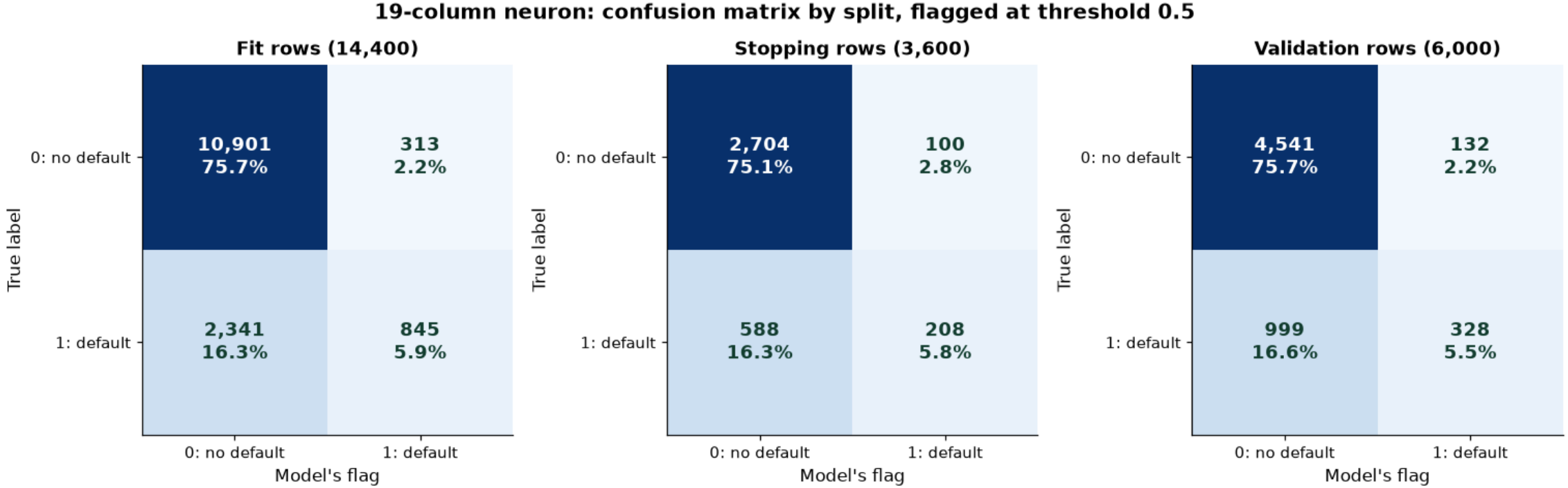
a risk grade or credit score

Validation, drawn



ROC AUC: neuron 0.7180, scikit-learn 0.7180, Course 4 0.7190. One model, one line.

Confusion matrices by split



Recall at 0.5: fit 0.2652, stopping 0.2613, validation 0.2472. The panels look alike.

Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(fit!A2:A14401)`	stats row 2, filled right: scaler.mean_
`=STDEV.P(fit!A2:A14401)`	stats row 3: scaler.scale_ (population)
`=(A2-stats!A$2)/stats!A$3`	valid U:AM: the fit rows' statistics
`=1/(1+EXP(-(stats!$B$6+SUMPRODUCT(U2:AM2,stats!$A$4:$S$4))))`	valid AO: predict
`=EXP(K2) and =EXP(K3)`	odds ratios 2.0571 and 0.9799

No Keras or network training in Excel. AVERAGE and STDEV.P equal scikit-learn's; outputs equal predict within 1e-6 on 6,000 rows.

Save the day: git diff

```
<hash> Experiment log: the 19-column neuron fitted in Keras, validation metrics
<hash> Experiment log as day 1 left it
<hash> netkit: fit_arrays and neuron_coefficients, 10 tests
```

```
+neuron_19_columns,validation,0.7180,0.4945,0.5000,0.8115,0.7130,0.2472,0.3671
```

One + line, no row removed: 4 decimals and LF line endings keep the diff clean.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the scaler is fitted on the fit and validation rows: means off 0 by 0.0085; LIMIT_BAL mean 166,784.12, not 166,552.78.

Code that runs is not the same as code that is right.

What you can do now

- Read a Dense layer's kernel and bias
- Scale with fit_arrays, on the fit rows only; see why scaling matters
- Show one neuron is scikit-learn's logistic regression
- Fit the same weights with Solver in Excel
- Read near-copy weights with care; choose on stopping rows

Week 1, Day 3 Hidden layers and activations